

CCNA Day 63

Ansible, Puppet, Chef

6.0 Automation and Programmability

10%



- 6.1 Explain how automation impacts network management
- 6.2 Compare traditional networks with controller-based networking
- 6.3 Describe controller-based and software defined architectures (overlay, underlay, and fabric)
 - 6.3.a Separation of control plane and data plane
 - 6.3.b North-bound and south-bound APIs
- 6.4 Compare traditional campus device management with Cisco DNA Center enabled device management
- 6.5 Describe characteristics of REST-based APIs (CRUD, HTTP verbs, and data encoding)
- 6.6 Recognize the capabilities of configuration management mechanisms Puppet, Chef, and Ansible
- 6.7 Interpret JSON encoded data

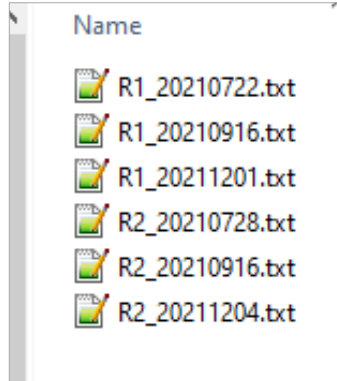


Things we'll cover

- Intro to Configuration Management Tools
- Ansible
- Puppet
- Chef

Configuration Drift

- *Configuration drift* is when individual changes made over time cause a device's configuration to deviate from the standard/correct configurations as defined by the company.
 - Although each device will have unique parts of its configuration (IP addresses, host name, etc), most of a device's configuration is usually defined in standard templates designed by the network architects/engineers of the company.
 - As individual engineers make changes to devices (for example to troubleshoot and fix network issues, test configurations, etc.), the configuration of a device can drift away from the standard.
 - Records of these individual changes and their reasons aren't kept.
 - This can lead to future issues.
- Even without automation tools, it is best to have standard *configuration management* practices.
 - When a change is made, save the config as a text file and place it in a shared folder.



- A standard naming system like *hostname_yyyymmdd* might be used.
- There are flaws to this system, as an engineer might forget to place the new config in the folder after making changes. Which one should be considered the 'correct' config?
- Even if configurations are properly saved like this, it doesn't guarantee that the configurations actually match the standard.

Configuration Provisioning

- *Configuration provisioning* refers to how configuration changes are applied to devices.
→ This includes configuring new devices, too.
- Traditionally, configuration provisioning is done by connecting to devices one-by-one via SSH.
→ This is not practical in large networks.
- Configuration management tools like Ansible, Puppet, and Chef allow us to make changes to devices on a mass scale with a fraction of the time/effort.
- Two essential components: *templates* and *variables*

```
hostname {{hostname}}
!
interface GigabitEthernet0/0
ip address {{address}} {{mask}}
ip ospf {{process_id}} area {{area}}
```

```
---
hostname: R1
address: 192.168.1.1
mask: 255.255.255.0
process_id: 1
area: 0
```

Template

Variables

Config

```
hostname R1
!
interface GigabitEthernet0/0
ip address 192.16.1.1 255.255.255.0
ip ospf 1 area 0
```

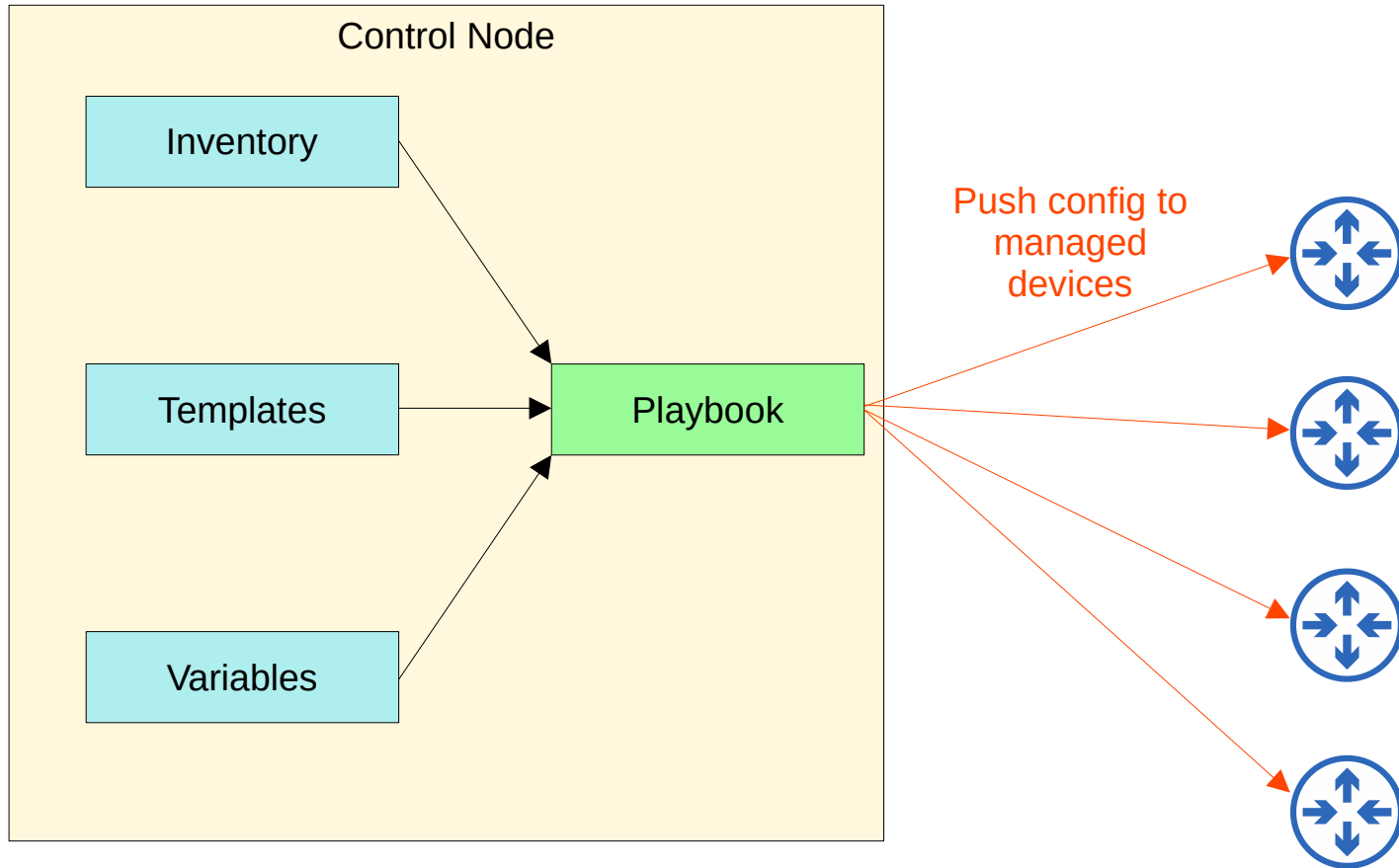
Configuration Management Tools

- *Configuration management tools* are network automation tools that facilitate the centralized control of large numbers of network devices.
- The options you need to be aware of for the CCNA are Ansible, Puppet, and Chef.
- These tools were originally developed after the rise of VMs, to enable server system admins to automate the process of creating, configuring, and removing VMs.
 - However, they are also widely used to manage network devices.
- These tools can be used to perform tasks such as:
 - Generate configurations for new devices on a large scale.
 - Perform configuration changes on devices (all devices in your network, or a certain subset of devices).
 - Check device configurations for compliance with defined standards.
 - Compare configurations between devices, and between different versions of configurations on the same device.



- Ansible is a configuration management tool owned by Red Hat.
- Ansible itself is written in Python.
- Ansible is *agentless*.
 - It doesn't require any special software to run on the managed devices.
- Ansible uses SSH to connect to devices, make configuration changes, extract information, etc.
- Ansible uses a *push* model. The Ansible server (Control node) uses SSH to connect to managed devices and *push* configuration changes to them.
 - Puppet and Chef use a *pull* model.
- After installing Ansible itself, you must create several text files:
 - **Playbooks**: These are files are 'blueprints of automation tasks'. They outline the logic and actions of the tasks that Ansible should do. Written in YAML.
 - **Inventory**: These files list the devices that will be managed by Ansible, as well as characteristics of each device such as their device role (access switch, core switch, WAN router, firewall, etc). Written in INI, YAML, or other formats.
 - **Templates**: These files represent a device's configuration file, but specific values for variables are not provided. Written in Jinja2 format.
 - **Variables**: These files list variables and their values. These values are substituted into the templates to create complete configuration files. Written in YAML.

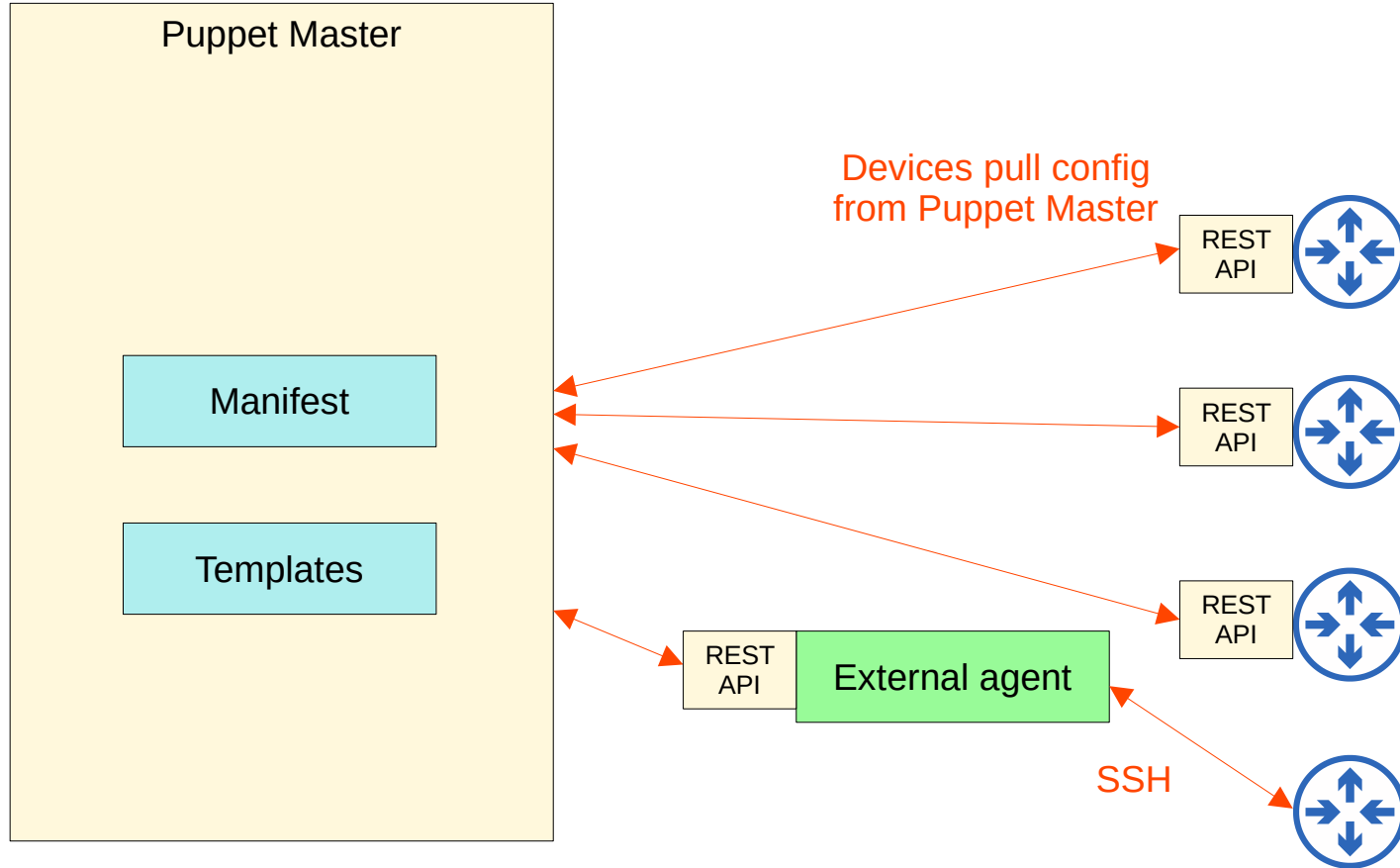




- Puppet is a configuration management tool written in Ruby.
- Puppet is typically agent-based.
 - Specific software must be installed on the managed devices.
 - Not all Cisco devices support a Puppet agent.
- It can be run agentless, in which a proxy agent runs on an external host, and the proxy agent uses SSH to connect to the managed devices and communicate with them.
- The Puppet server is called the 'Puppet master'.
- Puppet uses a pull model (clients 'pull' configurations from the Puppet master).
 - Clients use TCP 8140 to communicate with the Puppet master.
- Instead of YAML, it uses a proprietary language for files.
- Text files required on the Puppet master include:
 - **Manifest**: This file defines the desired configuration state of a network device.
 - **Templates**: Similar to Ansible templates. Used to generate Manifests.

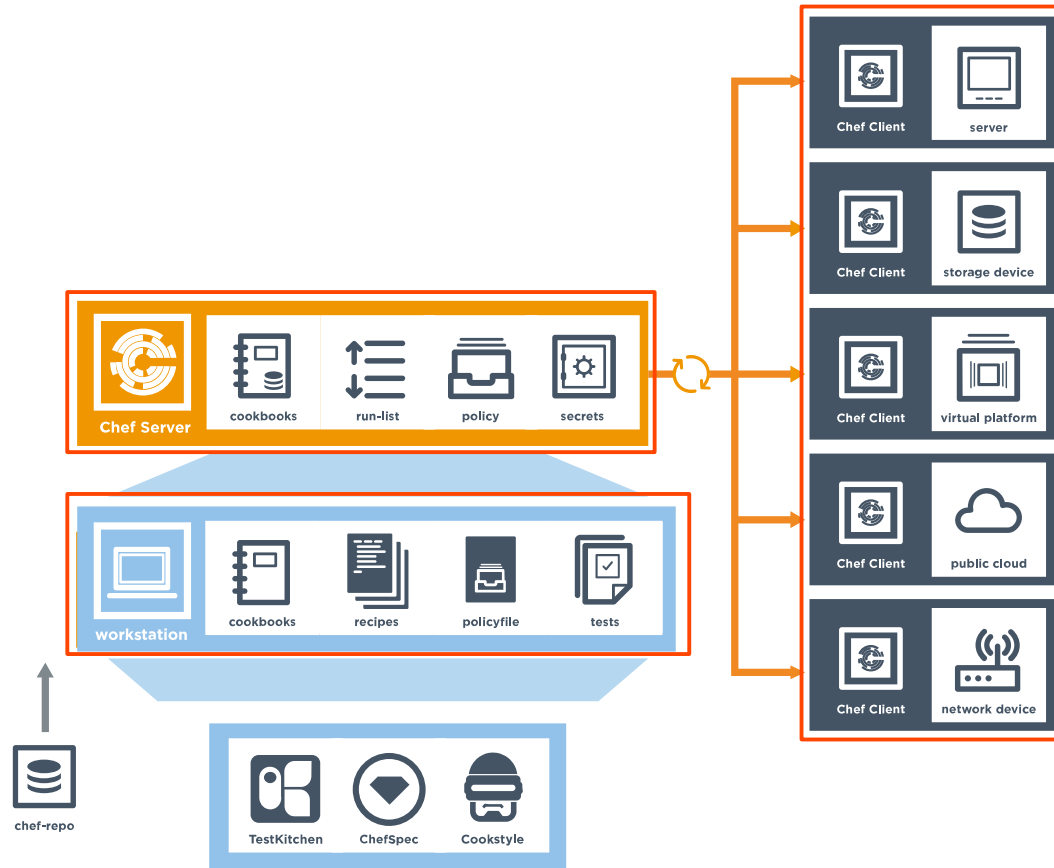


Puppet



- Chef is a configuration management tool written in Ruby.
- Chef is agent-based.
 - Specific software must be installed on the managed devices.
 - Not all Cisco devices support a Chef agent.
- Chef uses a pull model.
- The server uses TCP 10002 to send configurations to clients.
- Files use a DSL (Domain-Specific Language) based on Ruby.
- Text files used by Chef include:
 - **Resources**: The 'ingredients' in a recipe. Configuration objects managed by Chef.
 - **Recipes**: The 'recipes' in a cookbook. Outline the logic and actions of the tasks performed on the resources.
 - **Cookbooks**: A set of related recipes grouped together.
 - **Run-list**: An ordered list of recipes that are run to bring a device to the desired configuration state.





https://docs.chef.io/chef_overview/

Ansible, Puppet, Chef comparison

	Ansible	Puppet	Chef
Key Files defining actions	Playbook	Manifest	Recipe, Run-list
Communication Protocol	SSH	HTTPS (via REST API)	HTTPS (via REST API)
Key Port	22 (SSH port)	8140	10002
Agent-based/ Agentless	Agentless	Agent-based (or Agentless)	Agent-based
Push/Pull	Push	Pull	Pull

Things we covered

- Intro to Configuration Management Tools
- Ansible
- Puppet
- Chef

Which of the following configuration management tools connects to devices using SSH?

- a) Chef
- b) Ansible
- c) Puppet
- d) None of the above

Which of the following configuration management tools use a pull model? Select all that apply.

- a) Chef
- b) Ansible
- c) Puppet
- d) All of the above

Which of the following configuration management tools use a client-server model?

- a) Chef
- b) Ansible
- c) Puppet
- d) All of the above

Which of the following configuration management tools are written in Ruby? Select all that apply.

- a) Chef
- b) Ansible
- c) Puppet
- d) None of the above

Which of the following configuration management tools uses playbooks?

- a) Chef
- b) Ansible
- c) Puppet
- d) None of the above